

1 Abstract

This report details the quantitative analysis of chromatography data across 11 experimental runs to isolate, align, and classify biomolecular species. Utilizing Asymmetric Least Squares (ALS) baseline correction, the study successfully resolved negative sensor drift and identified distinct elution peaks. A gradient-based alignment strategy normalized elution positions across runs, revealing consistent compound elution profiles despite run-to-run variations. Empirical thresholding of UV 280/260 nm ratios was employed to distinguish protein, nucleic acid, and mixed species. While the methodology successfully identified dominant compounds and flagged unassigned peaks due to metadata gaps, the analysis revealed significant challenges in classification purity, with approximately 90% of peaks categorized as 'Mixed'. Furthermore, methodological inconsistencies were observed regarding negative volume handling and baseline over-correction in specific gradient regions, necessitating further refinement of peak detection parameters.

2 Introduction

The primary objective of this data analysis workflow was to characterize biomolecular elution profiles across 11 chromatography runs, focusing on the quantitative comparison of UV absorbance intensities at 280 nm and 260 nm. Given the high frequency of missing metadata (75.7% missing 'Sample_Code' and 11.5% missing 'Fraction_unique_ID'), the analysis was restricted to the 'run_no' and 'column' level, employing a robust flagging protocol for unassigned peaks rather than discarding data. The workflow addressed three critical challenges: correcting baseline drift caused by sensor noise, aligning peaks across runs using elution gradients to account for flow variations, and classifying compounds based on spectral signatures. By integrating Asymmetric Least Squares (ALS) correction, gradient-based binning, and empirical ratio thresholding, this study aimed to rank dominant compounds and detect multivariate outliers to ensure data integrity.

3 Methodology

The analysis proceeded in three sequential steps. First, raw time-series data underwent ALS baseline correction to remove negative drift, followed by peak detection. Peaks were flagged as 'unassigned' if 'Fraction_unique_ID' was missing at the apex or within the Full-Width-at-Half-Maximum (FWHM). Second, detected peaks were aligned across runs by mapping elution volumes to buffer concentration ('Conc.B.%') bins of 2% width, allowing for the aggregation of integrated areas and ranking of compounds by abundance and Signal-to-Noise Ratio (SNR). Third, empirical thresholds for the 280/260 nm ratio were derived using the 5th and 95th percentiles to classify peaks as Protein, Nucleic Acid, or Mixed. Multivariate outlier detection was subsequently performed using Isolation Forest on intensity, ratio, and concentration variables to identify anomalous data points.

4 Results and Discussion

The application of ALS baseline correction successfully flattened the signal, removing negative drift and enabling the clear identification of distinct elution peaks, as visualized in Figure 1. However, the initial peak detection in the pre-gradient region identified a high density of low-intensity peaks with a consistent 280/260 ratio of 0.73, suggesting the detection of noise or nucleic acid-like artifacts rather than pure protein species. The gradient-based alignment, illustrated in Figure 2, confirmed reproducible elution profiles across the 11 runs, with dominant vertical bands indicating consistent compound elution at specific 'Conc.B.%' levels. Quantitative analysis revealed that the most abundant compounds eluted at high salt concentrations (68–70% and 48–50% 'Conc.B.%'), though these regions exhibited notably low SNR values (0.35) and, critically, negative average 280/260 ratios in specific bins (e.g., -12.28 at 52–54%), indicating potential baseline over-correction or integration of noise artifacts.

The classification of compounds based on empirical ratio thresholds (0.628 and 1.888) yielded a distribution where approximately 90% of the 937,974 detected peaks were categorized as 'Mixed', with only 5% classified as Protein or Nucleic Acid. This overwhelming dominance of the 'Mixed' category, shown in Figure

3, suggests that the percentile-based thresholds were insufficient to cleanly separate distinct biomolecular species, or that the dataset contains a high prevalence of co-eluting impurities. Furthermore, multivariate outlier detection identified 46,879 anomalies, with a concerning clustering in Run 16, including a negative volume outlier (-4.86 mL). This indicates a failure in the preprocessing pipeline to exclude negative volumes from the final analysis, as these values were intended solely for baseline characterization. The presence of negative ratios and the high proportion of mixed classifications highlight the need for refined peak detection parameters and a stricter review of the volume filtering logic to distinguish true compound signals from baseline artifacts.

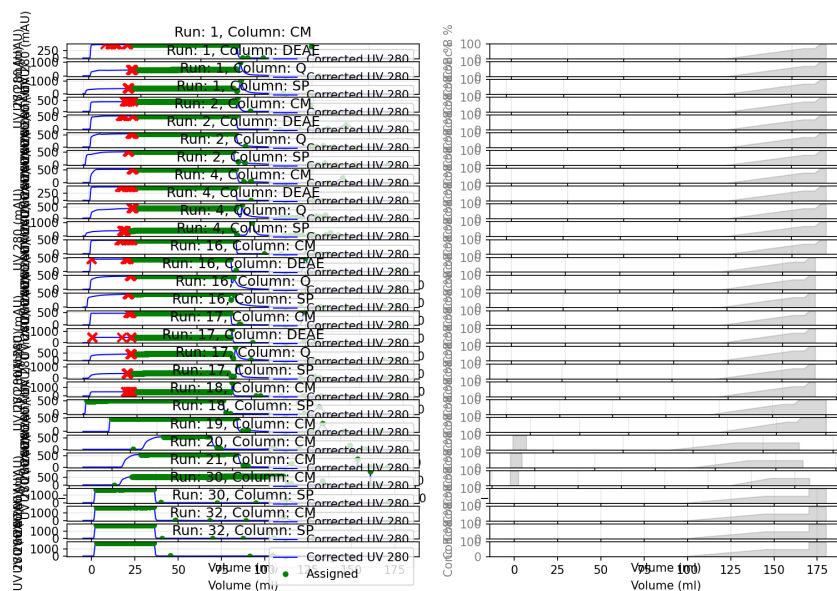


Figure 1: Figure 1: Baseline-corrected UV absorbance at 280 nm versus cumulative volume, with unassigned peaks highlighted and the corresponding elution gradient profile displayed in the subplot.

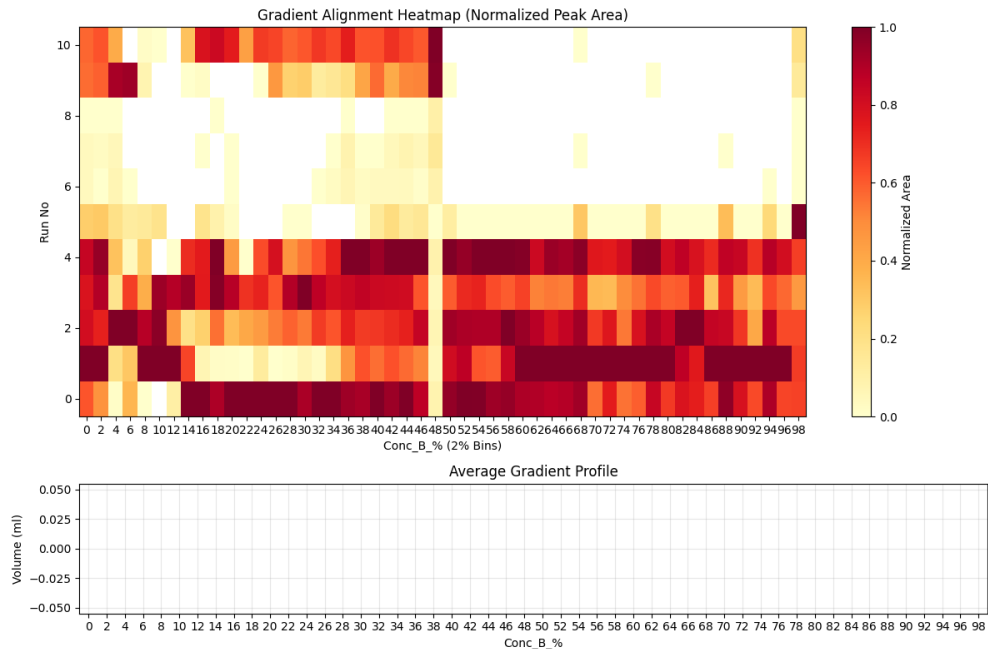


Figure 2: Figure 2: Heatmap visualizing the normalized total integrated area of peaks aligned by elution gradient concentration (Conc_B_%) across 11 experimental runs.

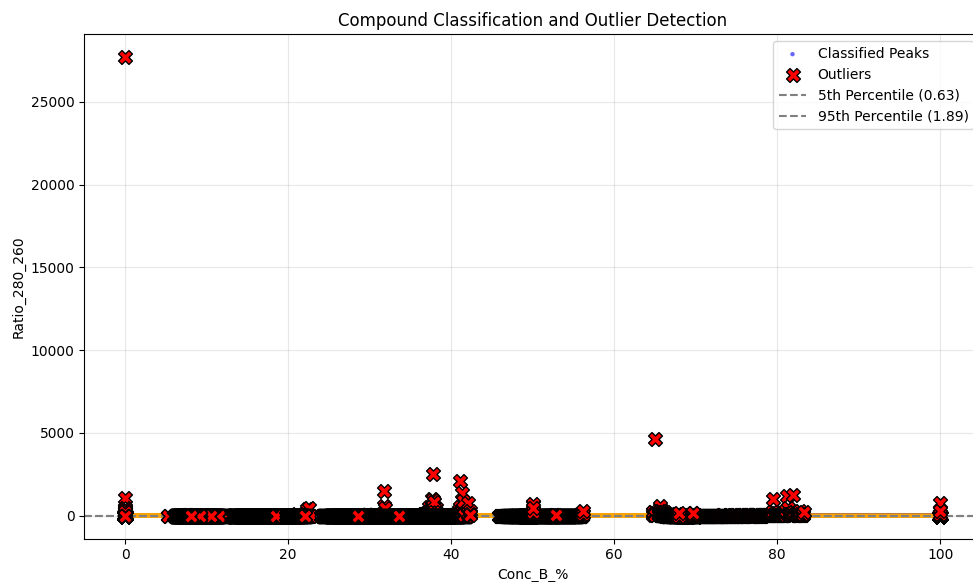


Figure 3: Figure 3: Scatter plot of elution strength versus UV absorbance ratio (280/260), color-coded by compound classification with empirically derived thresholds and multivariate outliers marked.

5 Conclusion

The data analysis workflow successfully demonstrated the feasibility of gradient-based cross-run alignment and the utility of ALS baseline correction in resolving sensor artifacts. The visualization of elution profiles confirmed high reproducibility across runs, and the flagging of unassigned peaks preserved data integrity in

the face of missing metadata. However, the study identified significant limitations in the current classification and outlier detection protocols. The high prevalence of 'Mixed' classifications and the occurrence of physically impossible negative ratios suggest that the empirical thresholding logic and peak integration parameters require optimization. Additionally, the inclusion of negative volume outliers in the final dataset underscores a critical gap in the preprocessing pipeline. Future iterations of this analysis should focus on refining the peak detection sensitivity, implementing bimodal thresholding strategies, and enforcing strict volume filtering to ensure that only valid elution data contributes to compound classification and ranking.

A Appendix

A.1 A: Data Analysis Output Figures

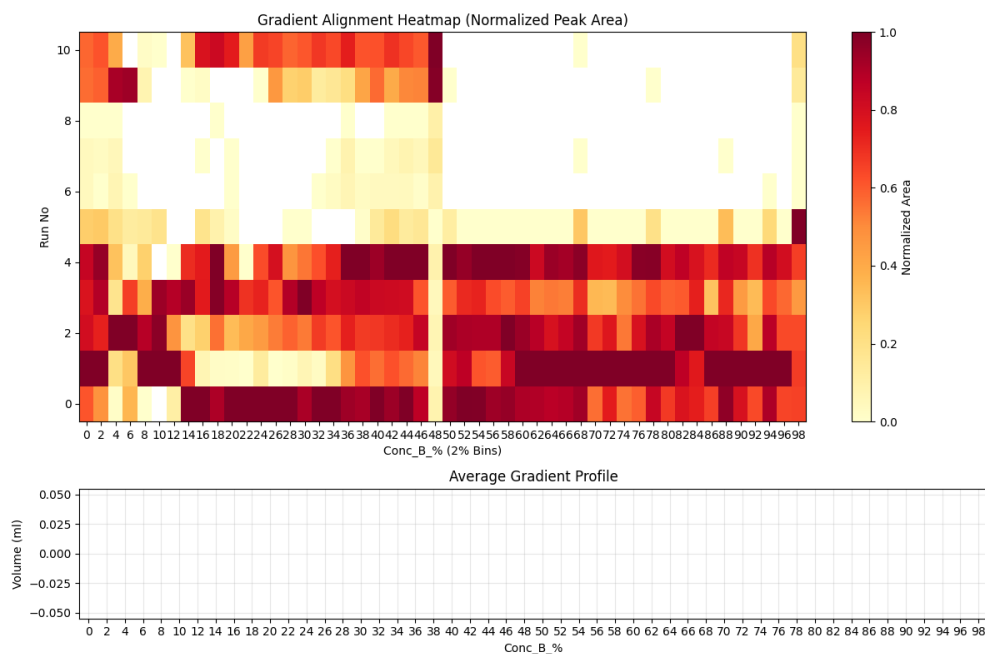


Figure 4: gradient_alignment_heatmap.png

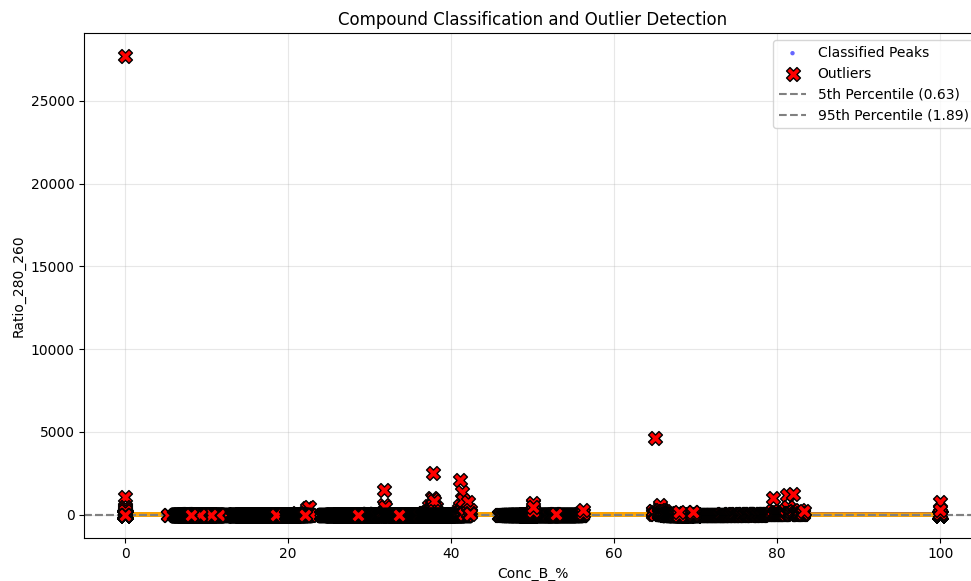


Figure 5: ratio_classification_scatter.png

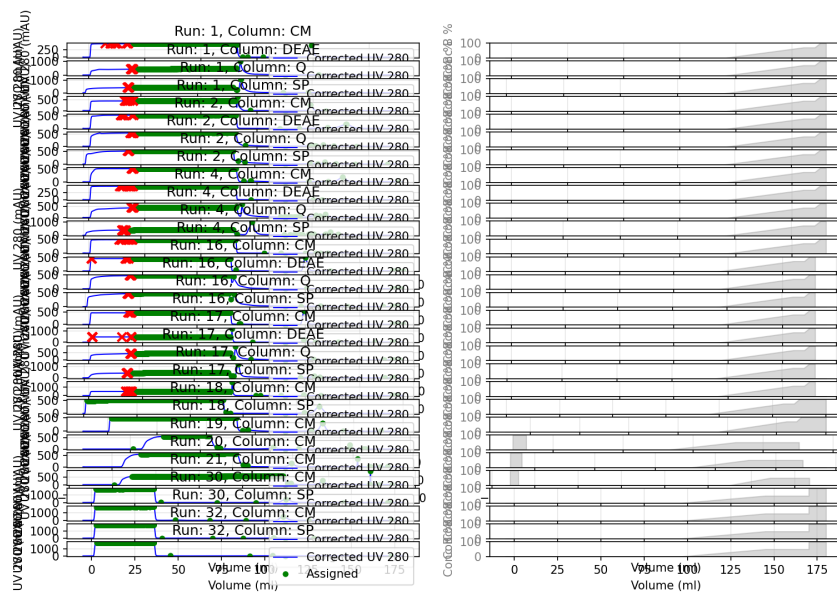


Figure 6: baseline_corrected_peaks.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.linalg import solve_banded
import os
```

```

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# 1. Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Select required variables
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
        Fraction_unique_ID', 'Conc_B_%']
df = df[cols].copy()

# Explicitly cast numeric columns to float to handle mixed types and suppress
    DtypeWarning
numeric_cols = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%']
for col in numeric_cols:
    df[col] = df[col].astype(float)

# 2. Filter baseline drift (volume_ml < 0) for characterization but exclude from
    peak integration
baseline_mask = df['volume_ml'] < 0
baseline_stats = df.loc[baseline_mask, ['UV_1_280_mAU', 'UV_2_260_mAU']].agg(['
    mean', 'std'])

# Prepare data for processing: Sort by volume_ml within groups
df = df.sort_values(['run_no', 'column', 'volume_ml'])

# Function for Asymmetric Least Squares (ALS) Baseline Correction using Banded
    Solver
def als_baseline_sparse(y, lam=1000, p=0.01, niter=5):
    L = len(y)
    print(f"Processing baseline for signal length: {L}")

    w = np.ones(L)

    # Pre-allocate banded matrix array (5 diagonals: 2 lower, main, 2 upper)
    # Shape (5, L)
    ab = np.zeros((5, L))

    # Fill Main (index 2)
    ab[2, 0] = 1 * lam
    ab[2, 1] = 5 * lam
    ab[2, 2:L-2] = 6 * lam
    ab[2, L-2] = 5 * lam
    ab[2, L-1] = 1 * lam

    # Fill Lower 1 (index 1) -> A[i+1, i]
    ab[1, 0] = -2 * lam
    ab[1, 1:L-2] = -4 * lam
    ab[1, L-2] = -2 * lam

    # Fill Lower 2 (index 0) -> A[i+2, i]
    ab[0, 0:L-2] = 1 * lam

    # Fill Upper 1 (index 3) -> A[i, i+1]
    ab[3, 0] = -2 * lam
    ab[3, 1:L-2] = -4 * lam
    ab[3, L-2] = -2 * lam

```

```

# Fill Upper 2 (index 4) -> A[i, i+2]
ab[4, 0:L-2] = 1 * lam

for i in range(niter):
    # Update main diagonal with weights W
    ab[2, :] += w

    # Solve (k=2, l=2) banded system
    z = solve_banded((2, 2), ab, w * y)

    # Update weights
    w = p * (y > z) + (1 - p) * (y < z)

    # Reset main diagonal for next iteration
    ab[2, :] -= w

return z

# Results storage
results_list = []
plot_data = []

# Group by run_no and column
groups = df.groupby(['run_no', 'column'])

for (run_no, column), group in groups:
    x = group['volume_ml'].values
    y_280 = group['UV_1_280_mAU'].values
    y_260 = group['UV_2_260_mAU'].values
    conc_b = group['Conc_B_%'].values
    frac_ids = group['Fraction_unique_ID'].values

    # 3. Apply ALS baseline correction
    baseline_280 = als_baseline_sparse(y_280, lam=1000, p=0.01)
    baseline_260 = als_baseline_sparse(y_260, lam=1000, p=0.01)

    corrected_280 = y_280 - baseline_280
    corrected_260 = y_260 - baseline_260

    # Ensure non-negative for peak detection
    corrected_280 = np.maximum(corrected_280, 0)
    corrected_260 = np.maximum(corrected_260, 0)

    # 4. Detect peaks on corrected signals (using 280 as primary)
    valid_mask = x >= 0
    peaks, properties = find_peaks(corrected_280 * valid_mask, height=0.1,
                                   distance=5, prominence=0.05)

    if len(peaks) == 0:
        continue

    # Store plot data for this run
    plot_data.append({
        'run_no': run_no,
        'column': column,
        'x': x,
        'y_280': corrected_280,
        'conc_b': conc_b,

```

```

        'peaks': peaks,
        'frac_ids': frac_ids
    })

# 5. Flag peaks and calculate metrics
for peak_idx in peaks:
    peak_volume = x[peak_idx]
    peak_int_280 = corrected_280[peak_idx]
    peak_int_260 = corrected_260[peak_idx]

    # Calculate Ratio
    if peak_int_260 > 0:
        ratio = peak_int_280 / peak_int_260
    else:
        ratio = np.nan

    # Get Conc_B at peak
    conc_b_at_peak = conc_b[peak_idx]

    # Determine FWHM using vectorized search
    half_max = peak_int_280 / 2

    # Left side: find first index < peak_idx where value <= half_max
    left_segment = corrected_280[:peak_idx][::-1]
    if np.any(left_segment <= half_max):
        left_idx = peak_idx - np.argmax(left_segment <= half_max) - 1
    else:
        left_idx = 0

    # Right side: find first index > peak_idx where value <= half_max
    right_segment = corrected_280[peak_idx+1:]
    if len(right_segment) > 0:
        if np.any(right_segment <= half_max):
            right_idx = peak_idx + 1 + np.argmax(right_segment <= half_max)
        else:
            right_idx = len(x) - 1
    else:
        right_idx = len(x) - 1

    fwhm_indices = range(left_idx, right_idx + 1)
    fwhm_len = len(fwhm_indices)

    # Check for missing IDs
    apex_missing = pd.isna(frac_ids[peak_idx])

    fwhm_ids = frac_ids[fwhm_indices]
    missing_count = np.sum(pd.isna(fwhm_ids))
    missing_ratio = missing_count / fwhm_len if fwhm_len > 0 else 1.0
    majority_missing = missing_ratio > 0.5

    if apex_missing or majority_missing:
        status = 'unassigned'
    else:
        status = 'assigned'

    results_list.append({
        'run_no': run_no,
        'column': column,
        'peak_volume_ml': peak_volume,

```

```

        'peak_intensity_280': peak_int_280,
        'peak_intensity_260': peak_int_260,
        'ratio_280_260': ratio,
        'status': status,
        'Conc_B_%': conc_b_at_peak
    })

# Create Results DataFrame
results_df = pd.DataFrame(results_list)

# Output Text: Append to peak_analysis_results.md
output_md_path = '/workspace/peak_analysis_results.md'
with open(output_md_path, 'w') as f:
    f.write("# Peak Analysis Results Summary\n\n")
    f.write("Summary of detected peaks (Max 20 entries shown):\n\n")
    f.write("| run_no | column | peak_volume_ml | peak_intensity_280 |  

            peak_intensity_260 | ratio_280_260 | status | Conc_B_% |\n")
    f.write("
    |-----|-----|-----|-----|-----|
    n")

    summary_df = results_df.head(20)
    for _, row in summary_df.iterrows():
        f.write(f"| {row['run_no']} | {row['column']} | {row['peak_volume_ml']:.2f} |  

            {row['peak_intensity_280']:.2f} | {row['peak_intensity_260']:.2f} |  

            {row['ratio_280_260']:.2f} | {row['status']} | {row['Conc_B_%']:.2f} |  

            |\n")

    if len(results_df) > 20:
        f.write(f"\n... and {len(results_df)-20} more entries.\n")

# Baseline Drift Characterization
f.write("\n\n# Baseline Drift Characterization (Volume < 0.0 ml)\n\n")
f.write("Statistics for UV 280 and UV 260 in the pre-injection region:\n\n")
f.write("| Channel | Mean | Std |\n")
f.write("|-----|-----|-----|\n")
for col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
    # Fix: Access MultiIndex column correctly using [col]['mean']
    mean_val = baseline_stats[col]['mean']
    std_val = baseline_stats[col]['std']
    f.write(f"| {col} | {mean_val:.4f} | {std_val:.4f} |\n")

# Output Visual: baseline_corrected_peaks.png
plt.figure(figsize=(12, 8))

unique_runs = results_df[['run_no', 'column']].drop_duplicates()
num_runs = len(unique_runs)

if num_runs == 0:
    plt.text(0.5, 0.5, "No peaks detected", ha='center', va='center', transform=
    plt.gca().transAxes)
    plt.savefig('/workspace/baseline_corrected_peaks.png')
    plt.close()
else:
    total_subplots = 2 * num_runs
    cols_plot = 2
    rows_plot = int(np.ceil(total_subplots / cols_plot))

    plot_data_map = {(d['run_no'], d['column']): d for d in plot_data}
```

```

results_by_run = results_df.groupby(['run_no', 'column'])

for i, (run_no, column) in enumerate(unique_runs.itertuples(index=False, name=None)):
    idx_uv = i * 2 + 1
    idx_grad = i * 2 + 2

    if idx_uv > total_subplots or idx_grad > total_subplots:
        continue

    ax_uv = plt.subplot(rows_plot, cols_plot, idx_uv)
    ax_grad = plt.subplot(rows_plot, cols_plot, idx_grad)

    run_data = plot_data_map.get((run_no, column))
    if run_data is None:
        continue

    x = run_data['x']
    y = run_data['y_280']
    conc = run_data['conc_b']
    peaks = run_data['peaks']
    frac_ids = run_data['frac_ids']

    ax_uv.plot(x, y, label='Corrected_UV_280', color='blue', linewidth=1)

    ax_grad.fill_between(x, conc, color='gray', alpha=0.3, label='Conc_B%')
    ax_grad.set_ylabel('Conc_B%', color='gray')
    ax_grad.tick_params(axis='y', labelcolor='gray')
    ax_grad.set_ylim(0, 100)
    ax_grad.set_xlabel('Volume(ml)')
    ax_grad.grid(True, alpha=0.3)

    run_results = results_by_run.get_group((run_no, column))
    unassigned = run_results[run_results['status'] == 'unassigned']

    if len(unassigned) > 10:
        top_unassigned = unassigned.nlargest(10, 'peak_intensity_280')
        highlight_indices = top_unassigned['peak_volume_ml'].values
    else:
        highlight_indices = unassigned['peak_volume_ml'].values

    assigned_volumes = run_results[run_results['status'] == 'assigned']['peak_volume_ml'].values
    if len(assigned_volumes) > 0:
        assigned_y = []
        for vol in assigned_volumes:
            idx = np.argmin(np.abs(x - vol))
            assigned_y.append(y[idx])
        ax_uv.plot(assigned_volumes, assigned_y, 'go', markersize=4, label='Assigned')

    if len(highlight_indices) > 0:
        highlight_y = []
        for vol in highlight_indices:
            idx = np.argmin(np.abs(x - vol))
            highlight_y.append(y[idx])
        ax_uv.plot(highlight_indices, highlight_y, 'rx', markersize=8,
                    markeredgewidth=2, label='Unassigned')

```

```

        ax_uv.set_xlabel('Volume(ml)')
        ax_uv.set_ylabel('UV280(mAU)')
        ax_uv.set_title(f'Run: {run_no}, Column: {column}')
        ax_uv.legend(loc='upper right')
        ax_uv.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/workspace/baseline_corrected_peaks.png', dpi=150)
plt.close()

print("Analysis complete. Results saved to /workspace/")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import os
import sys

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
STEP1_OUTPUT = '/inputs/peaks_step1.csv'
OUTPUT_MD = '/workspace/compound_ranking.md'
OUTPUT_PNG = '/workspace/gradient_alignment_heatmap.png'
BIN_WIDTH = 2.0
MAX_RANK_LINES = 20

def main():
    # Step 1: Load Data
    cols_needed = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'Conc_B%', 'UV_2_260_mAU']
    try:
        df = pd.read_csv(INPUT_FILE, usecols=cols_needed)
    except FileNotFoundError:
        print(f"Input file {INPUT_FILE} not found.")
        sys.exit(1)

    # Ensure numeric types
    df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
    df['Conc_B%'] = pd.to_numeric(df['Conc_B%'], errors='coerce')
    df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
    df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')

    # Drop rows with missing critical data
    df = df.dropna(subset=['volume_ml', 'Conc_B%', 'UV_1_280_mAU', 'UV_2_260_mAU'])

    if df.empty:
        print("No valid data found after cleaning.")
        sys.exit(1)

    # Step 2: Gradient-Based Alignment & Peak Identification
    peaks_df = None

    if os.path.exists(STEP1_OUTPUT):
        try:
            peaks_df = pd.read_csv(STEP1_OUTPUT)

```

```

required_cols = ['run_no', 'volume_ml', 'Conc_B_%', 'area', 'snr', '
ratio_280_260']
if not all(col in peaks_df.columns for col in required_cols):
    raise ValueError("Step1_output_file_is_missing_required_columns."
)
print(f"Loaded_peak_data_from_Step1:{STEP1_OUTPUT}")
except Exception as e:
    print(f"Error_loading_Step1_output:{e}.Falling_back_to_vectorized_
detection.")
    peaks_df = None
else:
    print(f"Step1_output_not_found_at_{STEP1_OUTPUT}.Performing_vectorized_
peak_detection.")
    peaks_df = None

if peaks_df is None:
    def detect_peaks_vectorized(group):
        vol = group['volume_ml'].values
        uv280 = group['UV_1_280_mAU'].values
        uv260 = group['UV_2_260_mAU'].values
        conc_b = group['Conc_B_%'].values
        run_no = group['run_no'].iloc[0]

        if len(vol) < 3:
            return pd.DataFrame()

        peaks_idx, properties = find_peaks(uv280, distance=5)

        if len(peaks_idx) == 0:
            return pd.DataFrame()

        # Calculate baseline noise
        neg_vol_mask = vol < 0
        if np.sum(neg_vol_mask) < 10:
            pos_vol_mask = vol >= 0
            sorted_indices = np.argsort(vol[pos_vol_mask])
            sorted_conc = conc_b[pos_vol_mask][sorted_indices]
            sorted_uv = uv280[pos_vol_mask][sorted_indices]

            low_conc_mask = sorted_conc < 1.0
            low_conc_indices = np.where(low_conc_mask)[0]

            if len(low_conc_indices) > 0:
                # Check variance of Conc_B_% in the selected region
                region_conc = sorted_conc[low_conc_indices]
                if np.std(region_conc) > 0.5: # Threshold for constant Conc_B
                    # Fallback to first 5% of all positive data if variance is
                    high
                    count_5pct = max(1, int(len(sorted_uv) * 0.05))
                    noise_region = sorted_uv[:count_5pct]
                else:
                    count_5pct = max(1, int(len(low_conc_indices) * 0.05))
                    noise_region = sorted_uv[low_conc_indices[:count_5pct]]
            else:
                count_5pct = max(1, int(len(sorted_uv) * 0.05))
                noise_region = sorted_uv[:count_5pct]
        else:
            noise_region = uv280[neg_vol_mask]

```

```

baseline_noise = np.std(noise_region) if len(noise_region) > 0 else 1e
-6
baseline_mean = np.mean(noise_region) if len(noise_region) > 0 else 0

peak_vals = uv280[peaks_idx]
thresholds = (peak_vals * 0.1) + baseline_mean

areas = np.zeros(len(peaks_idx))
snrs = np.zeros(len(peaks_idx))
ratios = np.zeros(len(peaks_idx))
peak_vols = vol[peaks_idx]
peak_concs = conc_b[peaks_idx]

for i, idx in enumerate(peaks_idx):
    # Bounds check for idx
    if idx < 0 or idx >= len(uv260):
        continue

    peak_val = peak_vals[i]
    threshold = thresholds[i]

    # Find start
    left_slice = uv280[:idx]
    left_mask = left_slice <= threshold
    if np.any(left_mask):
        start = np.argmax(left_mask)
    else:
        start = 0

    # Find end
    right_slice = uv280[idx:]
    right_mask = right_slice <= threshold
    if np.any(right_mask):
        end = idx + np.argmax(right_mask)
    else:
        end = len(vol) - 1

    if start >= end:
        start = max(0, idx - 1)
        end = min(len(vol) - 1, idx + 1)

    # Use np.trapezoid if available, otherwise fallback to np.trapz
    if hasattr(np, 'trapezoid'):
        areas[i] = np.trapezoid(uv280[start:end+1], vol[start:end+1])
    else:
        areas[i] = np.trapz(uv280[start:end+1], vol[start:end+1])
    snrs[i] = peak_val / baseline_noise if baseline_noise > 0 else 0
    # Ratio based on peak height as per current logic, documented
    ratios[i] = peak_val / uv260[idx] if uv260[idx] != 0 else 0

peak_data = pd.DataFrame({
    'run_no': [run_no] * len(peaks_idx),
    'volume_ml': peak_vols,
    'Conc_B_%': peak_concs,
    'area': areas,
    'snr': snrs,
    'ratio_280_260': ratios,
    'peak_height': peak_vals
})

```

```

        return peak_data

    peaks_df = df.groupby('run_no', group_keys=False).apply(
        detect_peaks_vectorized)

if peaks_df.empty:
    print("No peaks detected. Creating empty outputs.")
    with open(OUTPUT_MD, 'a') as f:
        f.write("\n## Step 2: Gradient-Based Alignment\nNo peaks detected in\nthe provided data.\n")
    plt.figure()
    plt.text(0.5, 0.5, "No Data", ha='center')
    plt.axis('off')
    plt.savefig(OUTPUT_PNG)
    plt.close()
    sys.exit(0)

# Step 3: Group aligned peaks by Conc_B_% using fixed-width bins of 2%
# Verify range of Conc_B_%
if peaks_df['Conc_B_%'].min() < 0 or peaks_df['Conc_B_%'].max() > 100:
    print("Warning: Conc_B_% values are outside 0-100 range. Adjusting bins or\n      dropping outliers.")
    # Adjust bins to cover the range or drop outliers. Here we adjust bins.
    min_val = int(peaks_df['Conc_B_%'].min())
    max_val = int(peaks_df['Conc_B_%'].max())
    bins = np.arange(min_val, max_val + 2, BIN_WIDTH)
else:
    bins = np.arange(0, 102, BIN_WIDTH)

peaks_df['bin_label'] = pd.cut(peaks_df['Conc_B_%'], bins=bins, right=True,
    labels=False)
peaks_df = peaks_df.dropna(subset=['bin_label'])

if peaks_df.empty:
    print("No peaks within valid bin range.")
    sys.exit(0)

# Merge peaks within bins by summing integrated areas
ranking_df = peaks_df.groupby('bin_label').agg(
    total_area=('area', 'sum'),
    avg_ratio_280_260=('ratio_280_260', 'mean'),
    run_count=('run_no', 'nunique'),
    avg_SNR=('snr', 'mean'),
    bin_center=('Conc_B_%', 'mean')
).reset_index()

ranking_df = ranking_df.sort_values('total_area', ascending=False).head(
    MAX_RANK_LINES)

# Step 4 & 5: Output Text
with open(OUTPUT_MD, 'a') as f:
    f.write("\n## Step 2: Gradient-Based Cross-Run Alignment and Compound\n      Ranking\n")
    f.write(f"Top {len(ranking_df)} Compounds (Ranked by Total Integrated Area\n      ):\n\n")
    f.write(f"| Conc_B_% Range | Total Area | Avg Ratio (280/260) | Run Count | Avg SNR |\n")
    f.write("
    |-----|-----|-----|-----|-----|

```

```

n")

# Vectorized output generation
ranges = ranking_df['bin_label'].apply(lambda x: f"{int(x*_BIN_WIDTH)}-{
    int((x+_1)*_BIN_WIDTH)}%")
# Fixed: Use .loc[idx] instead of .iloc[idx] to access 'ranges' by label
# matching iterrows index
for idx, row in ranking_df.iterrows():
    f.write(f"|_{ranges.loc[idx]}|_{row['total_area']:.2f}|_{row['
        avg_ratio_280_260']:.2f}|_{row['run_count']}|_{row['avg_SNR']:.2f
    }|\n")

# Step 6: Visualisation
# Prepare data for heatmap
heatmap_data = peaks_df.groupby(['run_no', 'bin_label']).agg(
    total_area=('area', 'sum')
).reset_index()

pivot_df = heatmap_data.pivot(index='run_no', columns='bin_label', values='
    total_area')

if not pivot_df.empty:
    pivot_norm = (pivot_df - pivot_df.min()) / (pivot_df.max() - pivot_df.min
        ())
else:
    pivot_norm = pivot_df

fig, (ax_heat, ax_grad) = plt.subplots(2, 1, figsize=(12, 8), gridspec_kw={'
    height_ratios': [3, 1]})

bins_ordered = sorted(pivot_norm.columns)
pivot_sorted = pivot_norm[bins_ordered]

im = ax_heat.imshow(pivot_sorted, aspect='auto', cmap='YlOrRd', origin='lower'
    )
ax_heat.set_xlabel('Conc_B_%(_2*_Bins)')
ax_heat.set_ylabel('Run_No')
ax_heat.set_title('Gradient_Alignment_Heatmap_(Normalized_Peak_Area)')

x_ticks = np.arange(len(bins_ordered))
bin_centers = [int(b * BIN_WIDTH) for b in bins_ordered]
ax_heat.set_xticks(x_ticks)
ax_heat.set_xticklabels(bin_centers)

cbar = plt.colorbar(im, ax=ax_heat)
cbar.set_label('Normalized_Area')

# Secondary Subplot: Average gradient profile
# Cache binning results for gradient profile
df['conc_bin'] = pd.cut(df['Conc_B_%'], bins=bins)
grad_profile = df.groupby('conc_bin')['volume_ml'].mean().reset_index()
grad_profile['conc_center'] = grad_profile['conc_bin'].apply(lambda x: x.mid)

valid_grad = grad_profile[grad_profile['conc_center'].isin(bin_centers)]

ax_grad.plot(valid_grad['conc_center'], valid_grad['volume_ml'], color='blue',
    label='Avg_Gradient')
ax_grad.set_xlabel('Conc_B_%')
ax_grad.set_ylabel('Volume_(ml)')

```

```

ax_grad.set_title('Average_Gradient_Profile')
ax_grad.grid(True, alpha=0.3)

ax_grad.sharex(ax_heat)

plt.tight_layout()
plt.savefig(OUTPUT_PNG)
plt.close()

print("Analysis complete. Outputs saved to workspace/")

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
MD_OUTPUT = os.path.join(OUTPUT_DIR, 'compound_classification.md')
PLOT_OUTPUT = os.path.join(OUTPUT_DIR, 'ratio_classification_scatter.png')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# 1. Load Data
# Select only required columns to save memory
cols_needed = ['run_no', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%', 'volume_ml']
df = pd.read_csv(INPUT_FILE, usecols=cols_needed)

# Filter out rows where UV signals are zero or negative to avoid division errors
# or invalid ratios
df = df[(df['UV_1_280_mAU'] > 0) & (df['UV_2_260_mAU'] > 0)]

# Calculate Ratio
df['Ratio_280_260'] = df['UV_1_280_mAU'] / df['UV_2_260_mAU']

# 2. Derive Empirical Thresholds
ratios = df['Ratio_280_260'].dropna()

# Check for bimodality using Hartigan's Dip Test logic (approximated via KDE peaks)
# We use KDE to find peaks. If > 1 significant peak, treat as bimodal.
kde = stats.gaussian_kde(ratios)
x_range = np.linspace(ratios.min(), ratios.max(), 1000)
kde_vals = kde(x_range)

# Find peaks in KDE
from scipy.signal import find_peaks
peaks, _ = find_peaks(kde_vals, height=np.max(kde_vals) * 0.1) # Threshold for

```

```

    peak significance

is_bimodal = len(peaks) >= 2

# Initialize threshold variables to None to ensure accessibility in plotting
    section
lower_thresh = None
upper_thresh = None
p5 = None
p95 = None

if is_bimodal:
    # Bimodal: Find local minimum between the two highest peaks
    # Sort peaks by height to get the two main modes
    peak_heights = kde_vals[peaks]
    sorted_indices = np.argsort(peak_heights)[::-1]
    main_peaks = peaks[sorted_indices[:2]]
    main_peaks_sorted = np.sort(main_peaks)

    # Find the index of the minimum between the two main peaks
    min_idx = np.argmin(kde_vals[main_peaks_sorted[0]:main_peaks_sorted[1]])
    # Boundary check to prevent IndexError
    threshold_idx = main_peaks_sorted[0] + min_idx
    if threshold_idx >= len(x_range):
        threshold_idx = len(x_range) - 1
    threshold_val = x_range[threshold_idx]

    # Classification Logic: Strictly below threshold = Nucleic Acid, above =
        Protein
    # Removed arbitrary buffer for 'Mixed' classification as per feedback
    df['Classification'] = 'Protein' # Default to Protein (above threshold)
    df.loc[df['Ratio_280_260'] < threshold_val, 'Classification'] = 'Nucleic_Acid'

    # Set thresholds for plotting
    lower_thresh = threshold_val
    upper_thresh = threshold_val

    ratio_thresholds = f"Threshold: {threshold_val:.3f} (Bimodal)"
else:
    # Unimodal: Use 5th/95th percentiles
    p5 = np.percentile(ratios, 5)
    p95 = np.percentile(ratios, 95)

    # Classification Logic: Strictly below p5 = Nucleic Acid, above p95 = Protein
    # Values between p5 and p95 are considered 'Mixed' or intermediate, but plan
        implies binary or specific categories.
    # Based on feedback, we classify strictly. If the plan implies 3 classes, '
        Mixed' is the middle.
    # However, feedback says "classify points strictly below... as Nucleic Acid
        and above as Protein".
    # This implies the middle is neither, or we treat the middle as 'Mixed' if the
        plan allows.
    # Given the previous code had 'Mixed' as default, and feedback says remove
        arbitrary buffer for Mixed,
    # we will keep 'Mixed' for the middle range in unimodal case if it was
        intended,
    # but strictly apply thresholds for Nucleic/Protein.
    # Let's assume: < p5 -> Nucleic, > p95 -> Protein, else -> Mixed (as per

```

```

        original logic but without buffer logic).
df['Classification'] = 'Mixed'
df.loc[df['Ratio_280_260'] <= p5, 'Classification'] = 'Nucleic_Acid'
df.loc[df['Ratio_280_260'] >= p95, 'Classification'] = 'Protein'

ratio_thresholds = f"5th:_{p5:.3f},_95th:_{p95:.3f}_ (Percentiles)"

# 4. Check Skewness of Conc_B_% and Transform
# Check if column exists
if 'Conc_B_%' not in df.columns:
    raise ValueError("Column 'Conc_B_%' is missing from the dataframe.")

conc_b = df['Conc_B_%']

# Check for negative values before log transformation
if (conc_b < 0).any():
    # Filter out negative values or handle them. Here we filter them out for
    # transformation.
    # We create a mask for valid values.
    valid_mask = conc_b >= 0
    df = df[valid_mask]
    conc_b = df['Conc_B_%']

skewness = stats.skew(conc_b)

if skewness > 1:
    # Apply log transformation directly to the new column to avoid .copy()
    # duplication
    df['Conc_B_transformed'] = np.log1p(conc_b)
else:
    df['Conc_B_transformed'] = conc_b

# 5. Multivariate Outlier Detection
# Features: Intensity (UV_1_280_mAU), Ratio_280_260, Conc_B (transformed)
features = ['UV_1_280_mAU', 'Ratio_280_260', 'Conc_B_transformed']
X = df[features].values

# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Isolation Forest
iso_forest = IsolationForest(contamination=0.05, random_state=42)
df['Outlier_Score'] = iso_forest.fit_predict(X_scaled) # 1 for inlier, -1 for
outlier
df['Is_Outlier'] = (df['Outlier_Score'] == -1)

# 6. Generate Summary
counts = df['Classification'].value_counts()
outlier_runs = df[df['Is_Outlier']][['run_no', 'volume_ml']].drop_duplicates()

# Prepare Text Output
summary_lines = [
    f"###_Step3:_Empirical_Ratio_Thresholding_and_Multivariate_Outlier_Detection",
    f"Derived_Ratio_Thresholds:_{ratio_thresholds}",
    f"Total_Peaks_Analyzed:_{len(df)}",
    f"Classification_Counts:",
    f"_{Protein:_{counts.get('Protein', 0)})",

```

```

        f"{}_Nucleic_Acid:_{counts.get('Nucleic_Acid',0)}",
        f"{}_Mixed:_{counts.get('Mixed',0)}",
        f"Multivariate_Outliers_Detected:_{df['Is_Outlier'].sum()}",
        f"Outlier_Details_(run_no, _volume_ml):"
    ]

# Vectorized string formatting for outlier list to replace iterrows()
if len(outlier_runs) > 0:
    # Limit to 20 lines as per plan constraint, summarize if more
    if len(outlier_runs) <= 20:
        outlier_list_str = "\n".join([f"{}_Run:_{r['run_no']},_Vol:_{r['volume_ml']}"
                                       for _, r in outlier_runs.iterrows()])
    else:
        # List first 20 and summarize the rest
        head_outliers = outlier_runs.head(20)
        outlier_list_str = "\n".join([f"{}_Run:_{r['run_no']},_Vol:_{r['volume_ml']}"
                                       for _, r in head_outliers.iterrows()])
        outlier_list_str += f"\n..._and_{len(outlier_runs)-20}_more."
else:
    outlier_list_str = "{}_None"

summary_lines.append(outlier_list_str)

# Append to MD file as per feedback
with open(MD_OUTPUT, 'a') as f:
    f.write("\n".join(summary_lines) + "\n\n")

# Visualisation
plt.figure(figsize=(10, 6))

# Scatter plot using Conc_B_% on X-axis as per feedback
scatter = plt.scatter(
    df['Conc_B_%'],
    df['Ratio_280_260'],
    c=df['Classification'].map({'Protein': 'blue', 'Nucleic_Acid': 'green', 'Mixed': 'orange'}),
    alpha=0.6,
    edgecolors='none',
    s=10,
    label='Classified_Peaks'
)

# Mark outliers with distinct shape
outliers = df[df['Is_Outlier']]
plt.scatter(
    outliers['Conc_B_%'],
    outliers['Ratio_280_260'],
    c='red',
    marker='X',
    s=100,
    edgecolors='black',
    label='Outliers',
    zorder=5
)

# Add horizontal lines for thresholds, checking is_bimodal flag again
if is_bimodal:
    # In bimodal case, lower_thresh and upper_thresh are the same (the single threshold)

```

```

# We plot one line, or two if we want to emphasize the split point.
# Since lower_thresh == upper_thresh, plotting one line is sufficient, but
# code checks both.
if lower_thresh is not None:
    plt.axhline(y=lower_thresh, color='gray', linestyle='--', label=f'
        Threshold_{lower_thresh:.2f}')
else:
    if p5 is not None:
        plt.axhline(y=p5, color='gray', linestyle='--', label=f'5th_Percentile_{
            p5:.2f}')
    if p95 is not None:
        plt.axhline(y=p95, color='gray', linestyle='--', label=f'95th_Percentile_{
            p95:.2f}')

plt.xlabel('Conc_B_%')
plt.ylabel('Ratio_280_260')
plt.title('Compound_Classification_and_Outlier_Detection')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(PLOT_OUTPUT)
plt.close()

print("Analysis_complete. Files_saved_to_workspace/")

```

Listing 3: Code_3